dbmigrate plugin

Instalación

Para usar, simplemente coloque el archivo zip (Descargar desde https://github.com/tdm00/cfwheels-dbmigrate-plugin) en su directorio de complementos y vuelva a cargar su aplicación.

Creando migraciones

Debe ver que la versión de la base de datos actuales 0 y las migraciones que ha creado deben aparecer en las migraciones disponibles.

Utilice el cuadro de selección para elegir una versión para migrar y haga clic en el botón Ir. Esto migrará la base de datos a esa versión y proporcionará retroalimentación sobre lo que se hizo.

El complemento DBMigrate también guardará el código SQL que se ejecutó en una carpeta denominada / db / sql / para su referencia.

Además, al ejecutar las migraciones (por ejemplo, en el entorno de diseño o de desarrollo), el código SQL generado también se guarda en archivos de texto. Es posible que prefiera utilizar estos para actualizar su base de datos de producción, en lugar de ejecutar el complemento DBMigrate en el servidor.

Advertencia: copia de seguridad de su base de datos primero! Este complemento está diseñado para manipular la estructura de la base de datos (incluyendo las tablas borradas!). El complemento se proporciona "tal cual" sin garantía de ningún tipo.

(Por razones de seguridad, también querrá asegurarse de que sus archivos de migración no estén disponibles para ser descargados directamente desde su servidor de producción).

Creando una tabla

Opción 1. Empezando desde una plantilla

Desplácese hasta la parte inferior de la página a la sección titulada "Crear un nuevo archivo de migración desde la plantilla". Seleccione la plantilla Crear tabla.

Puede seleccionar si desea utilizar el prefijo Timestamp (mejor cuando trabaja como un equipo) o el prefijo numérico (más simple y más legible). A continuación, introduzca una descripción de lo que hace la migración, por ejemplo, crea una tabla de miembros.

Si selecciona el prefijo numérico, se creará un archivo denominado 001_creates_members_table.cfc en la carpeta / db / migrate /.

Abra el archivo y cambie [tableName] a miembros.

Opción 2. Empezando desde un archivo en blanco

Primero, cree un archivo CFC de migración denominado 001_members.cfc en la carpeta / db / migrate /. Este CFC necesita extender plugins.dbmigrate.Migration

Proporcione un atributo descriptivo para la etiqueta para indicar qué hace esta migración.

Agregue dos funciones, una nombrada para realizar cambios en la base de datos (es decir, la creación de la tabla) y otra denominada para revertir esos cambios. A continuación, para cada función añadir algunas etiquetas.

Su archivo de migración vacía se verá así:

```
<cfcomponent extends="plugins.dbmigrate.Migration" hint="creates member table">
  <cffunction name="up">
    <cfscript>

    </cfscript>
  </cffunction>
  <cffunction name="down">
    <cfscript>

    </cfscript>
  </cffunction>
</cfcomponent>
```

Ahora en la función up vamos a añadir algún script para crear la tabla de miembros. Para empezar utilizamos la función createTable para crear un objeto de definición de tabla.

```
                t = createTable('members');
```

Una llave primaria llamada “id” será añadida por defecto.

Agregando columnas.

Ahora añadiremos columnas a la definición de la tabla, Primero agregamos una columna de tipo cadena (string) para almacenar la dirección de correo electrónico de los miembros:

```
                        t.string('email');
```

A continuación agregaremos columnas para el nombre de los miembros. Esta vez agregamos dos columnas juntas (usando una lista de nombres) y también especificamos un límite de caracteres en lugar de usar el valor por defecto de 255. Tenga en cuenta que estamos usando Camel Case para nombres de columna y también necesitamos ahora especificar nombres de argumento.

```
                t.string(columnNames='firstname,lastname',limit=50);
```

También almacenaremos una columna de contraseña

```
                t.string(columnNames='password',limit=32);
```

A continuación, queremos registrar si el miembro es un administrador de sitio utilizando una columna booleana. Esta vez también especificaremos un valor predeterminado para la columna e indicaremos que no queremos permitir valores nulos.

```
      t.boolean(columnNames='isadministrator',default=false,null=false);
```

Por último, queremos añadir algunas marcas de tiempo automatizadas para que CFWheels grabe automáticamente cuando se creó el registro (createdat), se actualizó por última vez (updatedat) y se estableció una marca de tiempo para la supresión de software (deletedat). Podemos hacer esto usando una sola función llamada [APITableDefinitionTimestamps timestamps].

```
                        t.timestamps();
```

Ahora tenemos nuestra definición de tabla, queremos confirmar estas instrucciones a la base de datos. Para las nuevas tablas hacemos esto utilizando la función [APITableDefinitionCreate create].

```
                        t.create();
```

Para la función “down” en la migración, queremos revertir este cambio (es decir, quitar la tabla de la base de datos.) Hacemos esto usando la función [APTIMigrableDropTable dropTable].

```
                        dropTable('members');
```

Así que nuestro archivo de migración finalmente se ve así:

```
<cfcomponent extends="plugins.dbmigrate.Migration" hint="creates member table">
  <cffunction name="up">
    <cfscript>
    t = createTable('members');
    t.string('email');
    t.string(columnNames='firstname,lastname',limit=50);
    t.string(columnNames='password',limit=32);
    t.boolean(columnNames='isadministrator',default=false,null=false);
    t.timestamps();
    t.create();
    </cfscript>
  </cffunction>
  <cffunction name="down">
    <cfscript>
    dropTable('members');
    </cfscript>
  </cffunction>
</cfcomponent>
```

Guarde su archivo y cargue su aplicación en un navegador. Debe ver que la versión de la base de datos actual es 0 y la migración de la tabla de miembros de crear debe aparecer en las migraciones disponibles.

Utilice el cuadro de selección para elegir esta migración y haga clic en el botón Ir. Esto migrará la base de datos a la versión 001 y proporcionará retroalimentación para decir que creó los miembros de la tabla.

Ahora intente migrar la base de datos de nuevo a la versión 0 y vuelva a comprobar la base de datos para confirmar que la tabla de miembros se ha eliminado.

Antes de continuar, vuelva a ejecutar la migración para volver a la versión 001.

Creando una tabla

En nuestro ejemplo de un sitio miembro, es posible que desee agregar funcionalidad de mailing list y permitir a los miembros suscribirse o cancelar la suscripción.

Empecemos creando un nuevo archivo de migración 003_mailing_lists.cfc y creando una tabla simple para almacenar mailing list names.

Aquí hay algún código para esto, al igual que la forma en que creamos la tabla de miembros en nuestra primera migración:

```
<cfcomponent extends="plugins.dbmigrate.Migration" hint="adds mailing lists and
member subscriptions">
  <cffunction name="up">
    <cfscript>
    t = createTable('mailinglists', id=true, primaryKey='idPrincipal');
    t.string('name');
    t.text('description');
    t.timestamps();
    t.create();
    </cfscript>
  </cffunction>
  <cffunction name="down">
    <cfscript>
    dropTable('mailinglists');
    </cfscript>
  </cffunction>
</cfcomponent>
```

A continuación crearemos una tabla de unión para almacenar suscripciones de miembros. Esta tabla necesitará hacer referencia tanto a las tablas de miembros como a las listas de correo. Como esto está relacionado con las mailingLists, tiene sentido colocarlo en la misma migración (no es necesario crear una nueva migración para cada tabla).

Esta nueva tabla, que llamaremos suscripciones, no necesariamente necesita una clave primaria. Así que comenzaremos a crear un nuevo objeto de definición de tabla sin una clave primaria:

T = createTable (nombre = 'suscripciones', id = false); A continuación utilizaremos la función de referencias para crear columnas memberid y mailinglistid. De forma predeterminada (a diferencia

de Rails Active Record), las restricciones de clave externa también se configurarán para los miembros (id) y las listas de correo (id).

T.referencias ('miembro, lista de correo'); A continuación, agregamos una columna para registrar cuando el miembro se suscribió a la lista de correo llamada createdat que será configurada automáticamente por CFWheels. A continuación, llame a la función create para ejecutar la creación de definición de tabla.

T.timestamp ('createdat'); T.create (); No te olvides de dejar caer esta mesa también. Debido a las restricciones de clave externa es necesario eliminar esta tabla antes de eliminar la tabla de listas de correo. El complemento DBMigrate también detectará las restricciones de clave externa y las eliminará antes de soltar la tabla.

DropTable ('suscripciones'); Así que nuestro archivo de migración final tiene este aspecto:

t = createTable('mailinglists'); t.string('name'); t.text('description'); t.timestamps(); t.create();

```
t = createTable(name='subscriptions',id=true, primaryKey='identificador');
referenceNames='mailinglists', reference='idPrincipal'
t.timestamp('createdat');
t.create();
</cfscript>
```

dropTable(‘subscriptions’);

```
dropTable('mailinglists');
</cfscript>
```

Guarde su archivo y cargue su aplicación en un navegador. Debería ver que esta nueva migración aparece en las migraciones disponibles.

Utilice el cuadro de selección para elegir esta migración y haga clic en el botón Ir. Esto migrará la base de datos a la versión 003 y proporcionará retroalimentación para decir que agregó las nuevas tablas y claves foráneas.

Ahora intente migrar la base de datos de nuevo a la versión 002 y vuelva a comprobar la base de datos para confirmar que se han eliminado las tablas. Siempre es bueno comprobar a medida que vaya que sus migraciones hacia abajo están funcionando.

Antes de continuar, vuelva a ejecutar la migración para volver a la versión 003.

A continuación, veremos la adición de índices de base de datos.

Creación de índices en columnas de tabla

En nuestro ejemplo de un sitio miembro, es posible que desee mejorar el rendimiento de buscar miembros “up” por su nombre de usuario en un nivel de base de datos mediante la adición de un índice.

También podemos aplicar en un nivel de base de datos que la columna de nombre de usuario que agregamos a la tabla de miembros en nuestra segunda migración es única.

Empecemos por crear un nuevo archivo de migración 004_member_indexes.cfc.

En nuestra función “up” usamos la función addIndex para crear el índice. Establezca el argumento único como verdadero.

```
        addIndex(table='members',columnNames='username',unique=true);
```

En nuestra función “down” queremos eliminar este índice mediante la función removeIndex. Podríamos haber especificado el nombre del índice como un argumento para addIndex pero por defecto será el nombre de la tabla seguido de un subrayado seguido del primer nombre de columna del índice.

```
        removeIndex(table='members',indexName='members_username');
```

Así que nuestro archivo de migración se ve así

```
<cfcomponent extends="plugins.dbmigrate.Migration" hint="unique member
usernames">
  <cffunction name="up">
    <cfscript>
    addIndex(table='members',columnNames='username',unique=true);
    </cfscript>
  </cffunction>
  <cffunction name="down">
    <cfscript>
    removeIndex(table='members',indexName='members_username');
    </cfscript>
  </cffunction>
</cfcomponent>
```

Guarde su archivo y cargue su aplicación en un navegador. Debería ver que esta nueva migración aparece en las migraciones disponibles.

Utilice el cuadro de selección para elegir esta migración y haga clic en el botón Ir. Esto migrará la base de datos a la versión 004 y proporcionará retroalimentación para decir que agregó el índice.

Ahora intente migrar la base de datos de nuevo a la versión 003 y vuelva a comprobar la base de datos para confirmar que se ha eliminado el índice.

Modificando una tabla existente

A medida que el proyecto se desarrolla, a menudo tendrá que realizar modificaciones en una tabla que ya ha creado (debido a cambios en las especificaciones o cosas que no previó cuando creó la tabla).

Si la migración de la tabla de creación ya se ha asignado a su repositorio de control de versiones y otros desarrolladores ya han ejecutado esa migración, no es ideal cambiar el código del archivo de migración original. En su lugar, crearemos un nuevo CFC de migración para realizar esos cambios.

Tal vez decida que desea que las personas se inicien con un nombre de usuario en lugar de su dirección de correo electrónico y que también desee registrar una columna de estado para cada miembro. Podemos usar la función changeTable para agregar estas columnas adicionales.

Comience creando un nuevo archivo CFC de migración denominado 002_member_changes.cfc en la carpeta / db / migrate. Dé a la migración una sugerencia como "modifica la tabla de miembros".

Su CFC de migración debería verse así:

```
<cfcomponent extends="plugins.dbmigrate.Migration" hint="modifies member table">
  <cffunction name="up">
    <cfscript>

    </cfscript>
  </cffunction>
  <cffunction name="down">
    <cfscript>

    </cfscript>
  </cffunction>
</cfcomponent>
```

A continuación, en la función up, obtenga un objeto de definición de tabla para la tabla de miembros utilizando la función changeTable.

```
                t = changeTable('members');
```

A continuación, agregue las dos nuevas columnas de la misma manera en que inicialmente agregamos columnas a la tabla.

```
        t.string(columnNames='username',limit=20);
        t.string(columnNames='status',limit=25);
```

Luego llama a la función de cambio para ejecutar estos cambios.

```
                t.change();
```

Tenga en cuenta que en lugar de utilizar la función changeTable, podríamos haber ejecutado dos llamadas de función addColumn separadas. p.ej.

```
addColumn(table='members',columnType='string',columnName='username',limit=20);
addColumn(table='members',columnType='string',columnName='status',limit=25);
```

Para la función “down” tenemos que revertir estos cambios con la función removeColumn.

```
removeColumn(table='members',columnName='username');
removeColumn(table='members',columnName='status');
```

Así que nuestro archivo de migración final tiene este aspecto:

```
<cfcomponent extends="plugins.dbmigrate.Migration" hint="modifies member table">
  <cffunction name="up">
    <cfscript>
    t = changeTable('members');
    t.string(columnNames='username',limit=20);
    t.string(columnNames='status',limit=25);
    t.change();
    </cfscript>
  </cffunction>
  <cffunction name="down">
    <cfscript>
    removeColumn(table='members',columnName='username');
    removeColumn(table='members',columnName='status');
    </cfscript>
  </cffunction>
</cfcomponent>
```

Guarde su archivo y cargue su aplicación en un navegador. Debería ver que esta nueva migración aparece en las migraciones disponibles.

Utilice el cuadro de selección para elegir esta migración y haga clic en el botón Ir. Esto migrará la base de datos a la versión 002 y proporcionará retroalimentación para decir que agregó las nuevas columnas.

Ahora intente migrar la base de datos de nuevo a la versión 001 y vuelva a comprobar la base de datos para confirmar que se han eliminado las columnas.

Antes de continuar, vuelva a ejecutar la migración para volver a la versión 002.

Migraciones de base de datos vía URL

A partir de la versión 0.8. de DBMigrate, ahora puede migrar su base de datos llamando a una URL

Para realizar migraciones a través de URL, necesitará usar el parámetro de URL de contraseña junto con la contraseña que ha especificado en su /config/settings.cfm para volver a cargar la aplicación (tiene una, verdad ???).

También necesitará utilizar el nuevo parámetro de URL migrateToVersion y especificar el número de versión de la migración a la que va a cambiar la base de datos.

Sintaxis

https:///rewrite.cfm?controller=wheels&action=wheels&view=plugins&name=dbmigrate&password=&migrateToVersion=99999999999999

• Utilice "migrateToVersion = 99999999999999" para migrar a la última versión de su base de datos.

• Utilice "migrateToVersion = 0" para eliminar todas las migraciones de su base de datos

• Utilice "migrateToVersion = 12345678912345" para migrar a esa versión de base de datos específica

Functions List

addColumn()

Descripción

Añade una columna a una tabla existente

Sintaxis

```
addColumn(table,columnType[,columnName,referenceName,default,null,limit,precision,scale]);
```

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción
Table	String	Si		Nombre de tabla existente

columnType	String	Si		Tipo de columna a ñadir(basado en lista de tipos de datos)
ColumnName	String	No		Nombre para nueva columna, requerida si columType no es ‘reference’
referenceName	string	No		Nombre para nueva columna de referencia
Default	String|integer	No		Valor predeterminado para la columna, ya sea una cadena o un entero dependiendo del tipo de datos
Null	Boolean	No		Si se permiten los nulos
Limit	Integer	No		Límite de tamaño de carácter o entero para la columna
Precisión	Integer	No		Valor de precisión para columnas decimales, es decir, número de dígitos que la columna puede contener
scale	Integer	No		El valor de la escala para las columnas decimales, es decir, el número de dígitos que se pueden colocar a la derecha del punto decimal (debe ser menor o igual a la precisión)

Ejemplos:

```
addColumn(table='members',columnType='string',columnName='status',limit=25);

addColumn(table='members',columnType='reference',referenceName='membertype');
```

addIndex()

Uso

Agregar índice de base de datos en una columna de tabla.

Sintaxis

```
addIndex(table,columnNames[,unique,indexName]);
```

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción
Table	String	Si		Nombre de la tabla para agregar índice
columnNames	String	Si		Uno o más nombres de columnas para indexar, separados por comas
unique	boolean	No	false	Si es verdadero creará una restricción de índice única
indexName	string	No	table name + underscore + first column name	Nombre para usar por index

Ejemplo:

```
addIndex(table='members',columnNames='username',unique=true);
```

addRecord()

Descripción

Genera sintaxis SQL para agregar un nuevo registro a una tabla de base de datos existente

Sintaxis

```
addRecord(table[[,columnName=columnValue\]\]\);
```

Parameters

Parámetro	Tipo	Requerido	Defecto	Descripción
Table	String	Si		Nombre de una tabla existente para agregar un nuevo registro a
Column(s)	String	Si		Lista de columnas y valores a insertar

Ejemplo

```
addRecord(table='members',id=1,username='admin',password='#Hash("admin")#');
```

Limites

Hay un número máximo de instrucciones addRecord que se pueden utilizar en una sola migración. Sin embargo, este límite es diferente dependiendo de factores como el hardware del servidor de aplicaciones, el hardware del servidor de bases de datos, la conectividad de red entre los dos y los valores de tiempo de espera que quizás se establezcan para todos estos sistemas. Las declaraciones de addRecord probadas máximas en una migración hasta la fecha eran 2486 en un sistema lento de desarrollo local. Se recomienda no tener más de 2000 declaraciones addRecord en un único archivo de migración.

announce()

Uso

Produce un comentario cuando se ejecuta la migración.

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción
Message	String	Si		Mensaje que se muestra cuando la migración es ejecutada

Ejemplo

```
        announce(message='I love database migrations!');
```

binary()

Uso

Añade una definición de columna binaria al objeto de definición de tabla

Sintaxis

```
                t.binary(columnNames[,default,null]);
```

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción
Message	String	Si		Mensaje que se muestra cuando la migración es ejecutada

default	String|integer	No		Valor por defecto para columna
null	boolean	No		whether nulls are allowed

boolean()

Uso

Agrega una definición de columna booleana al objeto de definición de tabla

Sintaxis

```
t.boolean(columnNames[,default,null]);
```

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción
columnNames	String	Si		Uno o más nombres de columna, delimitados por coma
default	String|integer	No		Valor por defecto para columna
null	boolean	No		whether nulls are allowed

changeColumn()

Uso

Cambia una definición de columna para una columna existente en una tabla

Sintaxis

```
changeColumn(table,columnName,columnType[,referenceName,default,null,limit,precision,scale,addColumns]);
```

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción
table	string	Si		Nombre de tabla existente encontrado en la base de datos
columnName	string	Si		Nombre de columna existente para cambiar
columnType	string	Si		Nuevo tipo para la columna
referenceName	string	No		Nombre para la nueva referencia de columna, requerido si el ColumnType es 'reference'
default	string	No		Valor determinado para la columna
null	boolean	No		Especifica si se permiten valores nulos en esta columna
limit	Integer	No		Límite de caracteres o tamaño entero para la columna
precision	Integer	No		Se utiliza para las columnas decimales para especificar el número de dígitos que la columna puede contener

scale	Integer	No		Número de dígitos que se pueden colocar a la derecha del punto decimal
addColumns	Boolean	No		Si se establece en true, los intentos de agregar columnas y bases de datos probablemente arrojarán un error si la columna ya existe

Ejemplo

```
changeColumn(table='members',columnType='string',columnName='status',limit=50);
```

changeTable()

Uso

Crea un objeto de definición de tabla para aplicar varios cambios de definición de columna a una tabla existente.

Sintaxis

```
t = changeTable(name);
```

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción
name	String	Si		Nombre de la tabla existente

Notas:

Tenga en cuenta que la tabla no se modificará en la base de datos hasta que llame a la función de cambio en el objeto de definición de tabla.

Si ya existe un nombre de columna en la tabla, se cambiará la definición de columna, de lo contrario se agregará la columna.

Si sólo desea agregar o cambiar una sola columna en la tabla, puede ser más sencillo utilizar las funciones addColumn, changeColumn o renameColumn.

Ejemplos:

```
t = changeTable('members');
// ... add some column definition changes ...
t.change();
```

change()

Uso

Cambia las columnas de la base de datos en base a las definiciones de columna almacenadas en esta TableDefinition creado por changeTable.

Sintaxis

```
t = changeTable(name);

// ... one or more column definitions here ...
t.change([addColumns]);
```

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción
addColumns	Boolean	No	False	Si es cierto, intentará agregar nuevas columnas de lo contrario comprobará si existe una columna para determinar si se agrega o actualiza

Notas:

Si addColumns se establece en true y la columna ya existe, su base de datos probablemente lanzará un error.

column()

Uso

Agrega una definición de columna al objeto de definición de tabla

Sintaxis

```
t.column(columnName,columnType[,default,null,limit,precision,scale]);
```

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción
columnName	String	Yes	False	Si es cierto, intentará agregar nuevas columnas de lo contrario comprobará si existe una columna para determinar si se agrega o actualiza
columnType	String	Yes		Tipo de columna, tipos permitidos incluyen primaryKey, [binary](apitabledefinitionbinary),

				[boolean](apitabledefinitionboolean), [date](apitabledefinitiondate), [datetime](apitabledefinitiondatetime), [decimal](apitabledefinitiondecimal), [float](apitabledefinitionfloat.md), [integer](apitabledefinitioninteger), [string](apitabledefinitionstring), [text](apitabledefinitiontext), [time](apitabledefinitiontime), [timestamp](apitabledefinitiontimestamp)
default	String|integer	No		Valor predeterminado para la columna, ya sea una cadena o un entero dependiendo del tipo de datos
null	Boolean	No		Si se permiten los nulos
limit	Integer	No		Límite de tamaño de carácter o entero para la columna
precision	Integer	No		Valor de precisión para columnas decimales, es decir, número de dígitos que la columna puede contener
scale	Integer	No		El valor de la escala para las columnas decimales, es decir, el número de dígitos que se pueden colocar a la derecha del punto decimal (debe ser menor o igual a la precisión)

Notas:

Es probable que probablemente no utilice esta función directamente, sino que probablemente desee utilizar las diferentes funciones de alias para diferentes tipos de datos (por ejemplo, cadena, texto, entero, etc.).

create()

Uso

Crea la tabla en la base de datos, utilizando las definiciones de columna proporcionadas al objeto de definición de tabla. Si se estableció el parámetro fuerza al crear la definición de tabla, la tabla (si ya existe) se eliminará primero.

Sintaxis

```
t = createTable(name);
// ... one or more column definitions here ...
t.create();
```

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción
None	string	yes		Uno o más nombres de columna, delimitados por coma

createTable()

Uso

Crea un objeto de definición de table

Sintaxis

```
t = createTable(name[,force,id,primaryKey]);
```

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción
name	string	yes		Nombre de la tabla
force	Boolean	No	False	Si o no a la caída de la tabla del mismo nombre antes de crear uno nuevo
id	Boolean	No	True	Si es false, define una tabla sin clave primaria
primaryKey	String	No	Id	Reemplaza el nombre de la llave primaria predeterminada

Notas

Tenga en cuenta que la tabla no se creará en la base de datos hasta que llame a la función de creación en el objeto de definición de tabla.

Ejemplos

```
// basic create table
t = createTable('members');

// force table to be dropped, if exists, before creating
t = createTable(name='members',force=true);

// create table with no primary key
// (e.g. can be used for hasMany relationship joining tables)
t = createTable(name='membermailinglists',id=false);

// create a table with a different primary key name
t = createTable(name='members',primaryKey='memberId');
```

createView()

Uso

Crea un objeto de definición de vista

Sintaxis

```
v = createView(name,);
```

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción
Name	string	yes		Nombre de la vista que quieres crear

Notas

Tenga en cuenta que la vista no se creará en la base de datos hasta que se llame a la función create en el objeto de definición de vista.

Ejemplos

```
// basic create ciew
v = createView('vUsers');
v.selectStatement("SELECT userid, (FirstName + ' ' + LastName) as fullName FROM
users");
v.create();
```

date()

Uso

Agrega una definición de columna de fecha al objeto de definición de tabla

Sintaxis

```
t.date(columnNames[,default,null]);
```

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción
columnNames	string	yes		Uno o más nombres de columna, delimitados por coma
default	String|integer	No		Valor predeterminado para la columna
null	boolean	no		Si se permiten los nulos

Notas

Algunas bases de datos no pueden almacenar una fecha sola y utilizarán un tipo de datos datetime.

datetime()

Uso

Agrega una definición de columna de fecha y hora al objeto de definición de tabla

Sintaxis

```
t.datetime(columnNames[,default,null]);
```

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción
columnNames	string	yes		Uno o más nombres de columna, delimitados por coma
default	String|integer	No		Valor predeterminado para la columna
null	boolean	no		Si se permiten los nulos

decimal()

Uso

Agrega una definición de columna decimal al objeto de definición de tabla

Sintaxis

```
t.decimal(columnNames[,default,null,precision,scale]);
```

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción
columnNames	string	yes		Uno o más nombres de columna, delimitados por coma
default	String|integer	No		Valor predeterminado para la columna
null	boolean	no		Si se permiten los nulos
precision	number	No		Valor de precisión para columnas decimales, es decir, número de dígitos que la columna puede contener
scale	integer	No		El valor de la escala para las columnas decimales, es decir, el número de dígitos que se pueden colocar a la derecha del punto decimal (debe ser menor o igual a la precisión)

char()

Uso

Agrega una definición de columna char al objeto de definición de tabla

Sintaxis

```
t.char(columnNames);
```

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción
columnNames	string	yes		Uno o más nombres de columna, delimitados por coma
null	boolean	no		Si se permiten los nulos
precision	number	No		Valor de precisión para columnas decimales, es decir, número de dígitos que la columna puede contener
scale	integer	No		El valor de la escala para las columnas decimales, es decir, el número de dígitos que se pueden colocar a la derecha del punto decimal (debe ser menor o igual a la precisión)

numeric()

Uso

Agrega una definición de columna numeric al objeto de definición de tabla

Sintaxis

```
t.numeric(columnNames[null]);
```

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción
columnNames	string	yes		Uno o más nombres de columna, delimitados por coma
null	boolean	no		Si se permiten los nulos

money()

Uso

Agrega una definición de columna money al objeto de definición de tabla

Sintaxis

```
t.money(columnNames[null]);
```

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción
columnNames	string	yes		Uno o más nombres de columna, delimitados por coma
null	boolean	no		Si se permiten los nulos

dropForeignKey()

Uso

Eliminar una restricción de clave externa de la base de datos.

Sintaxis

```
dropForeignKey(table,keyName);
```

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción
table	string	si		Nombre de la tabla
keyName	boolean	Si		Nombre de la llave foránea

Ejemplo

```
dropForeignKey(table='members',keyName='FK_members_membertypes');
```

dropReference()

Uso

Eliminar una restricción de clave externa de la base de datos, utilizando el nombre de referencia que se utilizó para crearlo.

Sintaxis

```
dropReference(table,referenceName);
```

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción
table	string	si		Nombre de la tabla existente
referenceName	string	Si		Nombre de la referencia que se utilizó para crear la restricción de clave externa
Campo	String	Si	id	Es el campo al que se hizo la referencia

Ejemplo

```
dropReference(table='members',referenceName='membertype', campo=’idPrincipal’);
// note this is really just an alias for
dropForeignKey(table='members',keyName='FK_members_membertypes');
```

dropTable()

Uso

Eliminar una tabla de una base de datos

Sintaxis

```
dropTable(name);
```

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción
name	string	si		Nombre de la tabla a eliminar

Ejemplo

```
dropTable('members');
```

dropView()

Uso

Eliminar una vista de una base de datos

Sintaxis

```
dropView(name);
```

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción

name	string	si		Vista a eliminar

Ejemplo

```
dropView('members');
```

execute()

Uso

Ejecuta una consulta SQL cruda, generalmente utilizada para funciones específicas de la base de datos.

Sintaxis

```
execute(sql);
```

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción
sql	string	si		Instrucción SQL a ejecutar

Ejemplo

```
<cfcomponent extends="plugins.dbmigrate.Migration" hint="add composite primary
key to tagging table">
  <cffunction name="up">
    <cfscript>
      execute('ALTER TABLE mydb.taggings ADD PRIMARY KEY (tagid,callid);');
    </cfscript>
  </cffunction>
  <cffunction name="down">
    <cfscript>
      execute('ALTER TABLE mydb.taggings DROP PRIMARY KEY');
    </cfscript>
  </cffunction>
</cfcomponent>
```

float()

Uso

Agrega una definición de columna flotante al objeto de definición de tabla

Sintaxis

```
t.float(columnNames[,default,null]);
```

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción
columnNames	string	si		Uno o más nombres de columnas delimitados por comas
default	Integer|float	No		Nombre por defecto para la columna
null	boolean	no		Valores nulos son permitidos

integer()

Uso

Agrega una definición de columna de número entero al objeto de definición de tabla

Sintaxis

```
t.integer(columnNames[,default,null,limit]);
```

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción
columnNames	string	si		Uno o más nombres de columnas delimitados por comas
default	Integer|float	No		Nombre por defecto para la columna
null	boolean	no		Valores nulos son permitidos
limit	integer	no		Límite de tamaño entero para la columna

primaryKey()

Uso

Agrega una definición de clave principal a la tabla. Este método también permite múltiples claves primarias

Sintaxis

```
t.primaryKey(name,[,type,autoIncrement,limit,precision,scale,references,onUpdate
,onDelete]);
```

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción
name	string	Si		Nombre de la columna de llave primaria
type	String	No		Tipo de columnas, incluyendo llave primaria, [binary](apitabledefinitionbinary), [boolean](apitabledefinitionboolean), [date](apitabledefinitiondate), [datetime](apitabledefinitiondatetime), [decimal](apitabledefinitiondecimal), [float](apitabledefinitionfloat.md), [integer](apitabledefinitioninteger), [string](apitabledefinitionstring), [text](apitabledefinitiontext), [time](apitabledefinitiontime), [timestamp](apitabledefinitiontimestamp)
autoIncrement	boolean	No		
limit	Integer	No		Límite de tamaño de carácter o entero para la columna
precision	Integer	No		Valor de precisión para columnas decimales, es decir, número de

				dígitos que la columna puede contener
scale	Integer	No		El valor de la escala para las columnas decimales, es decir, el número de dígitos que se pueden colocar a la derecha del punto decimal (debe ser menor o igual a la precisión)
References	String	No		Tabla, esta clave primaria debe hacer referencia como una clave externa
onUpdate	String	No		Cómo desea que la restricción actúe en la actualización. Los posibles valores incluyen `none`,` null` o `cascade`, que también se puede establecer en` true`.
onDelete	String	No		Cómo desea que la restricción actúe al eliminar. Los posibles valores incluyen `none`,` null` o `cascade`, que también se puede establecer en` true`

references()

Uso

Agrega la definición de columna al objeto de definición de tabla para una columna de asociación (belongsTo, hasOne, hasMany) usando las convenciones CFWheels. Opcionalmente, agrega una restricción de clave externa.

Sintaxis

```
t.references(referenceNames[,reference]);
```

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción
referenceNames	string	si		Nombre de la tabla
reference	string	si		Nombre del campo para hacer la referencia

Ejemplos

```
t.references('user');
```

Creará una columna denominada userid y una restricción de clave externa al campo id de la tabla user.

```
t.references(referenceNames='user',reference='idPrincipal');
```

Creará una columna con referencia a la tabla user con referencia al campo idPrincipal

removeColumn()

Uso

Eliminar una columna de una tabla

Sintaxis

```
removeColumn(table[,columnName,referenceName]);
```

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción
table	string	si		Nombre de la tabla existente
columnName	String	No		Nombre de la columna existente
referenceName	string	no		Nombre de la referencia que se utilizó para crear la columna

Ejemplos

```
removeColumn(table='members',columnName='status');

removeColumn(table='members',referenceName='membertype');
```

removeIndex()

Uso

Eliminar índice de base de datos en una columna de tabla.

Sintaxis

```
removeIndex(table,indexName);
```

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción
table	string	si		Nombre de la tabla
indexName	String	si		Nombre del index a eliminar

Ejemplos

```
removeIndex(table='members',indexName='members_username');
```

removeRecord()

Uso

Elimina uno o más registros de una tabla.

Sintaxis

```
removeRecord(table[,where]);
```

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción

table	string	si		Nombre de la tabla para eliminar registros
Where	String	no		condición

Ejemplos

```
removeRecord(table='members',where='id=1');

removeRecord(table="members",where="lastname='smith' AND firstname='john'");
```

renameColumn()

Uso

Renombrar una columna existente en una tabla

Sintaxis

```
renameColumn(table,columnName,newColumnName);
```

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción
table	string	si		Nombre de la tabla existente
columnName	string	Si		Nombre de la columna existente
newColumnName	String	si		Nuevo nombre para la columna

Ejemplos

```
renameColumn(table='members',columnName='status',newColumnName='memberStatus');
```

renameTable()

Uso

Renombrar una tabla existente en una base de datos

Sintaxis

```
renameTable(oldName,newName);
```

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción
oldName	string	si		Nombre actual de la tabla
newName	string	Si		Nuevo nombre de la tabla

Ejemplos

```
renameTable(oldName='members',newName='users');
```

string()

Uso

Agrega una definición de columna de cadena al objeto de definición de tabla

Sintaxis

```
t.string(columnNames[,default,null,limit]);
```

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción
columnNames	string	si		Uno o más nombres de columna, delimitados por coma
default	Integer|float	No	null	Valor por defecto para la columna
null	Boolean	No		Si se permiten valores nulos
limit	integer	no		Límite de tamaño de caracteres para la columna

Ejemplos

```
t.string(columnNames="familyname",default="",null=true,limit="255");
```

text()

Uso

Agrega una definición de columna de texto al objeto de definición de tabla

Sintaxis

```
t.text(columnNames[,default,null]);
```

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción
columnNames	string	si		Uno o más nombres de columna, delimitados por coma
default	Integer|float	No	null	Valor por defecto para la columna
null	Boolean	No		Si se permiten valores nulos

Notas

Es posible que algunas bases de datos no permitan establecer un valor predeterminado para columnas con tipo de datos de texto.

time()

Uso

Agrega una definición de columna de tiempo al objeto de definición de tabla

Sintaxis

```
t.time(columnNames[,default,null]);
```

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción
columnNames	string	si		Uno o más nombres de columna, delimitados por coma
default	Integer|float	No	null	Valor por defecto para la columna
null	Boolean	No		Si se permiten valores nulos

Notas

Algunas bases de datos no pueden almacenar un tiempo a solas y utilizarán un tipo de datos de fecha y hora en su lugar.

timestap()

Uso

Agrega una definición de columna timestamp al objeto de definición de tabla

Sintaxis

```
t.timestamp(columnNames[,default,null]);
```

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción

columnNames	string	si		Uno o más nombres de columna, delimitados por coma
default	Integer|float	No	null	Valor por defecto para la columna
null	Boolean	No		Si se permiten valores nulos

timestaps()

Uso

Agrega 3 definiciones de columna de fecha y hora (createdat, updatedat, deletedat) al objeto de definición de tabla siguiendo las convenciones CFWheels para timestamps automatizados y borrados suaves.

Sintaxis

```
t.timestamps();
```

updateRecord()

Uso

Actualiza un registro en una tabla

Sintaxis

```
updateRecord(table[,where][[,columnName=columnValue]]);
```

Parámetros

Parámetro	Tipo	Requerido	Defecto	Descripción
table	string	si		Nombre de la tabla a actualizar registros
where	string	No		Cuando condición
one or more columnNames	string	No		Usar nombre de columna como nombre de argumento y valor

Ejemplo

```
updateRecord(table='members',where='id=1',status='Active');
```